

Systemdokumentation & Anwenderhandbuch

Raum-zeitliche Analyse und interaktive Visualisierung der ÖPNV-Versorgungsqualität in der Region Karlsruhe

Projektkontext: Studienprojekt im Kurs "Data Science & Web Engineering"

Studiengang: Onlinemedien (6. Fachsemester)

Hochschule: Duale Hochschule Baden-Württemberg (DHBW) Mosbach

Bearbeiter: Elia Küstner

Akzentfarben: KVV-Signalrot (#E3000B) & Muted Blaugrau (#C7CFE3)

Datum: Juni 2026

Version: 1.4.0 (Final Release)

1. Einleitung & Projektrahmen

Die vorliegende Systemdokumentation beschreibt die Konzeption, Architektur und Implementierung der Webanwendung **ÖPNV-Analytics Karlsruhe**. Im Rahmen des Onlinemedien-Studiums an der DHBW Mosbach wurde eine datengetriebene Fullstack-Anwendung entwickelt, die eine hochauflösende raumzeitliche Analyse der öffentlichen Verkehrsversorgung im Stadt- und Landkreis Karlsruhe ermöglicht. Ziel des Systems ist es, demografische Rasterdaten mit diskreten Fahrplänen und Einrichtungen der Daseinsvorsorge zu verschneiden, um Versorgungsdisparitäten transparent offenzulegen.

Herkömmliche Visualisierungen von Verkehrsnetzen beschränken sich häufig auf rein geografische Liniennetzpläne, welche die tatsächliche zeitliche Komponente (Angebotsfrequenz) sowie den realen Bevölkerungsbezug vernachlässigen. ÖPNV-Analytics löst diese Restriktion durch ein quadratisches Analyse-Grid von **3 ext{km} imes 3 ext{km}** Kacheln ab. Jede Kachel fungiert als eigenständige Analyseeinheit, in der demografische Zensusdaten mit den Fahrplanfrequenzen des Karlsruher Verkehrsverbundes (KVV) aggregiert und mit Geodaten wichtiger Points of Interest (POIs) verknüpft werden.

Design-Prämisse: Das gesamte System folgt einer strikten *Modern Minimalist / Flat Design* Ästhetik. Es wird konsequent auf abgerundete Ecken, komplexe UI-Verläufe und dekorative UI-Patterns verzichtet. Klare geometrische Kanten, präzise typografische Hierarchien und der selektive Einsatz der KVV-Signalfarbe Corporate Red steuern die Benutzerführung.

2. Systemarchitektur & Technologiestack

Die Software implementiert eine klassische dreischichtige Architektur (3-Tier-Architecture), um Datenhaltung, serverseitige Aggregationslogik und clientseitige Präsentation strikt voneinander zu trennen und eine hohe Wartbarkeit zu gewährleisten.

2.1 Datenschicht (Data Tier)

Als relationales Datenbank-Management-System (RDBMS) kommt eine lokale **PostgreSQL** Instanz zum Einsatz. Diese speichert sowohl die hochauflösenden Geometriedaten der Bevölkerung als auch das komplexe relationale GTFS-Fahrplanskelett und die kommunalen POI-Tabellen. Durch den Verzicht auf rechenintensive PostGIS-Geometrie-Abfragen zur Laufzeit und die stattdessen gewählte Vorberechnung im ETL-Kern wird eine hervorragende Abfragegeschwindigkeit erzielt.

2.2 Logikschicht (Logic Tier / Backend)

Das serverseitige Backend ist vollständig in **Python 3.11+** realisiert. Die Bereitstellung der REST-Schnittstelle erfolgt über das moderne, asynchrone Web-Framework **FastAPI**. Der Applikationsserver wird durch **Uvicorn** betrieben. Die datenbankseitigen Abfragen sind über **SQLAlchemy Core** hochgradig optimiert, während komplexe mathematisch-geometrische Umprojektionen und räumliche Verschneidungen vorab über **GeoPandas** und **Shapely** prozessiert werden.

2.3 Präsentationsschicht (Presentation Tier / Frontend)

Das clientseitige Dashboard ist als schlanke Single-Page-Application (SPA) in nativem **Vanilla JavaScript (ES6+)**, HTML5 und CSS3 umgesetzt. Es kommen zwei spezialisierte Visualisierungs-Engines zum Einsatz:

- **Leaflet.js (v1.9.4)**: Zuständig für das performante Rendern des unbestechlichen Kachelnetzes. Über die Aktivierung des systemseitigen HTML5-Canvas-Modus (`preferCanvas: true`) werden alle Polygone als native Vektorpfade im Browser gezeichnet, wodurch Latenzen beim Verschieben und Zoomen eliminiert werden.
- **Chart.js (v4.x)**: Generiert das interaktive 24-Stunden-Linienprofil des Fahrplantaktes. Ein maßgeschneidertes, mathematisch kalibriertes Plugin sorgt für die exakte Einblendung der Pendler-Kernzeiten im Hintergrund des Diagrammrahmens.

3. Datenbankdesign & Relationales Schema

Das System verarbeitet drei primäre Datenströme, die statisch und relational über eine berechnete Primärschlüssel-ID miteinander korreliert sind. Die zentrale Schaltstelle für das Frontend bildet die Tabelle `kachel_analytics`.

Tabelle: kachel_analytics (Relationales Zieldesign)

Spaltenname	Datentyp	Funktion & Beschreibung
kachel_id	SERIAL (PK)	Eindeutiger System-Identifikator der 3 ext{km} imes 3 ext{km} Rasterzelle.
x_min / y_min	DOUBLE PRECISION	Mathematische linke untere Kante im europäischen ETRS89-LAEA Metrisch-System (EPSG:3035).
einwohner	INTEGER	Summierte Einwohnerzahl aller im Quadrat liegenden Zensus-Zellen.
bevoelkerungs_klasse	VARCHAR(50)	Strukturelle Zoneneinstufung (Ländlich bis Metropolitan) basierend auf absoluten Schwellenwerten.
p1_lat / p1_lon	DOUBLE PRECISION	Präzise GPS-Koordinaten (WGS84 / EPSG:4326) der Ecke unten-links für Leaflet.
p2_lat bis p4_lon	DOUBLE PRECISION	Analoge GPS-Eckpunkte für unten-rechts, oben-rechts und oben-links.
anzahl_haltestellen	INTEGER	Gesamtzahl aller KVV-Haltestellen innerhalb dieses Rasterquadrats.
linien_liste	TEXT	Alphabetisch sortierte, kommasetrennte Liste aller kreuzenden Bus- und Bahnlinien.
takt_pendler_morgens	DOUBLE PRECISION	Ø Abfahrten pro Stunde im exakten Zeitfenster von 06:30 bis 08:30 Uhr an Werktagen.
takt_wochenende	DOUBLE PRECISION	Ø Abfahrten pro Stunde am Samstag und Sonntag (48h-Gesamtbasis).
takt_24h_array	TEXT	Kommasetrennter String bestehend aus exakt 24 Gleitkommawerten (Stunde 00:00 bis 23:00 Uhr).
dist_hospital_km	DOUBLE PRECISION	Metrisch exakte Luftliniendistanz zum nächstgelegenen Krankenhaus.
nearest_hospital_name	VARCHAR(255)	Offizieller Name des geografisch nächstgelegenen Krankenhauses.
dist_townhall_km	DOUBLE PRECISION	Luftliniendistanz zum nächsten Rathaus/Bürgerbüro (kommunale Daseinsvorsorge).

4. Der ETL-Prozess (backend/pipeline.py)

Das Skript `pipeline.py` bildet das mathematische Herzstück des Backends. Es operiert als eigenständiges ETL-Modul (Extract, Transform, Load) und löst zwei fundamentale Herausforderungen der Geoinformatik: die Detektion und Behebung des systematischen Koordinatenversatzes sowie das lückenlose Schließen topologischer Fragmente.

4.1 Behebung des Zensus-Zentroiden-Versatzes

Die Rohdaten des Zensus 2022 liefern die Koordinaten der Bevölkerung in Form von Zellenmittelpunkten (Zentroiden), definiert durch die Spalten `x_mp_1km` und `y_mp_1km` im Koordinatenreferenzsystem EPSG: 3035. Würde man auf diese Mittelpunkte direkt eine klassische Ganzzahldivision zur Aggregation auf das **3 ext{km}**-Raster anwenden, führt dies zu einer asymmetrischen Verschiebung des Gitternetzes. Die Zellen würden mathematisch "gefleort", was das Gitter auf der Leaflet-Karte um bis zu zwei Kilometer nach Norden verschiebt.

Die Pipeline löst dies unbestechlich, indem sie von jedem Mittelpunkt exakt **500\, ext{m}** subtrahiert. Dadurch wird die mathematische Unterkante der Zelle berechnet, bevor die Aggregationsstufe greift:

```
# Der mathematische Fix für absolute Positionstreuung auf der Karte
df_zensus['x_left'] = df_zensus['x_mp_1km'] - 500
df_zensus['y_bottom'] = df_zensus['y_mp_1km'] - 500

df_zensus['x_3k'] = (df_zensus['x_left'] // 3000) * 3000
df_zensus['y_3k'] = (df_zensus['y_bottom'] // 3000) * 3000
df_grid_base = df_zensus.groupby(['x_3k', 'y_3k']).agg({'einwohner':
'sum'}).reset_index()
```

4.2 Das Schließen struktureller Löcher via Convex Hull

Ein reiner `INNER JOIN` zwischen Zensus-Quadraten und Haltestellen führt dazu, dass Kacheln, die zwar besiedelt sind, aber keine physische Haltestelle direkt im Quadrat besitzen (z. B. Waldgebiete, landwirtschaftliche Flächen oder reine Wohngebiete ohne direkte Netzanbindung), als weiße Löcher aus der Karte gerissen werden.

Die Pipeline eliminiert dieses Problem durch den Einsatz eines geometrischen Mantels. Es werden alle Haltestellen und POIs im Zielgebiet geladen, in ein einheitliches metrisches System projiziert und ein stabiler, geschlossener Polygon-Mantel (**Convex Hull**) um die gesamte Punktwolke gespannt. Jede Zensus-Kachel, die diesen dichten Mantel schneidet, bleibt erhalten. Dadurch wird das Netz vollständig geschlossen und behält dennoch die organische Struktur des Landkreises bei:

```
valid_points = [Point(r['x_3k'] + 1500, r['y_3k'] + 1500) for _, r in
df_valid_cells.iterrows()]
region_mantel = MultiPoint(valid_points).convex_hull

# Filterung: Behalte alle Kacheln, die die Netzhülle schneiden
gdf_grid =
gdf_grid_all[gdf_grid_all.geometry.intersects(region_mantel)].reset_index(drop=True)
```

5. Die REST-API Schnittstelle (backend/main.py)

Die FastAPI-Anwendung stellt die performante Schnittstelle zur Verfügung. Um Performance-Glitches zu verhindern, nutzt die API das performante `row._mapping`-Verfahren von SQLAlchemy. Dadurch wird sichergestellt, dass die Datenbankspalten entkoppelt von ihrer Indexreihenfolge ausgelesen werden.

5.1 Endpunkt für Kartendaten

```
GET /api/kacheln?indicator={indicator}
```

Dieser Endpunkt liefert ein kompaktes JSON-Array aller Kacheln für das Leaflet-Rendering. Um Bandbreite zu sparen, werden hier exakt nur die ID, die vier WGS84-GPS-Ecken sowie der konkrete Wert des ausgewählten Indikators übergeben:

```
[
  {
    "kachel_id": 42,
    "p1_lat": 48.9936, "p1_lon": 8.3821,
    "p2_lat": 48.9942, "p2_lon": 8.4215,
    "p3_lat": 49.0211, "p3_lon": 8.4222,
    "p4_lat": 49.0205, "p4_lon": 8.3828,
    "value": 24500.0
  }
]
```

5.2 Endpunkt für Kachel-Details

```
GET /api/kachel/{kachel_id}
```

Sobald ein Benutzer eine Rasterzelle anklickt, feuert dieser Endpunkt und liefert das vollständige Datenblatt der Zelle inklusive des kommagetrennten Fahrplan-Strings für das Frontend-Diagramm.

6. Das interaktive Frontend-Dashboard

Das Frontend implementiert die Anwendungslogik und sorgt für eine verzögerungsfreie Interaktion zwischen Karte und Dashboard-Sidebar.

6.1 Absolutes Zensus-Zonensystem mit Opacity-Kopplung

Um die demografische Struktur des Landkreises realitätsgetreu abzubilden, implementiert die Funktion `getColorForScale` die exakten Schwellenwerte der Zensus-Zonierung. Gleichzeitig steuert die Applikation die Sichtbarkeit über ein dynamisches Transparenz-System (`getOpacityForScale`): Dünn besiedelte ländliche Räume werden blass und unaufdringlich im Hintergrund gerendert, während urbane Zentren in deckendem Knallgrün hervorstechen:

- **Unbesiedelte Zone (0 Einw.):** Transparentes Blaugrau (`#eef1f6`), Opacity: 0.15
- **Ländliche Zone (1 - 4.500 Einw.):** Hellrot (`#ffb3b3`), Opacity: 0.40
- **Aussenstädtische Zone (> 4.500 - 13.500 Einw.):** Blasses Orange (`#ffd9b3`), Opacity: 0.55
- **Urbane Kernzone (> 13.500 - 36.000 Einw.):** Hellgrün (`#a3ffa3`), Opacity: 0.75
- **Metropolitane Kernzone (> 36.000 Einw.):** Knallgrün (`#00c832`), Opacity: 0.90

6.2 Stufenlose Farb-Interpolation für Haltestellen

Wählt der Nutzer den Indikator "Anzahl Haltestellen", schaltet das Frontend auf eine mathematisch stufenlose RGB-Linear-Interpolation um. Die Kacheln morphen nahtlos je nach relativer Dichte zwischen Hellrot (Gering) und Knallgrün (Hoch):

```
const val = value / maxVal;
const r = Math.round(255 + (0 - 255) * val);
const g = Math.round(179 + (200 - 179) * val);
const b = Math.round(179 + (50 - 179) * val);
return `rgb(${r},${g},${b})`;
```

6.3 Ausfallsichere Pixel-Interpolation im Linienchart

Da die X-Achse des 24h-Profiles Text-Labels verwendet (`00:00` bis `23:00`), schaltet Chart.js intern in den Kategorie-Modus. Ein direkter Aufruf von Pixelwerten für Halbstunden-Schritte (wie 06:30 Uhr) führt unweigerlich zum Absturz des Skripts. Das integrierte `coreHoursPlugin` löst dies durch eine unbestechliche lineare Interpolation auf Basis der nativen Ticks:

```
const getXPixel = (hour) => {
  const integerPart = Math.floor(hour);
  const fractionalPart = hour - integerPart;
  const p1 = x.getPixelForValue(integerPart);
  const p2 = integerPart < 23 ? x.getPixelForValue(integerPart + 1) : p1;
  return p1 + (p2 - p1) * fractionalPart;
};
```

7. Installations- & Setup-Anleitung

Befolgen Sie diese Schritte, um das Gesamtsystem auf einem lokalen Windows-Entwicklungsrechner in Betrieb zu nehmen.

Schritt 1: Umgebungskonfiguration (.env)

Erstellen Sie im Projekt-Hauptverzeichnis eine Datei namens `.env` mit folgenden Variablen:

```
DB_HOST=127.0.0.1
DB_PORT=5432
DB_NAME=karlsruhe_transport
DB_USER=postgres
DB_PASSWORD=dein_geheimes_passwort
```

Schritt 2: Virtual Environment & Abhängigkeiten einrichten

Öffnen Sie ein PowerShell-Terminal im Projektverzeichnis und führen Sie folgende Befehle aus:

```
# 1. Virtuelle Umgebung initialisieren
python -m venv venv

# 2. Umgebung aktivieren
.\venv\Scripts\activate

# 3. Pip-Pakete installieren
pip install fastapi uvicorn pandas geopandas shapely sqlalchemy psycopy2 python-dotenv
```

Schritt 3: Datenpipeline ausführen

Vor dem ersten Start der Webanwendung muss die Analysetabelle initial berechnet und in die PostgreSQL-Datenbank geschrieben werden:

```
python backend/pipeline.py
```


Das Skript meldet nach erfolgreicher Berechnung: 🎉 ETL-Pipeline erfolgreich beendet! . Die Tabelle umfasst genau 221 bereinigte Kacheln.

8. Bedienungsanleitung & Troubleshooting

8.1 Feature-Walkthrough im Betrieb

Starten Sie das System durch einen Doppelklick auf die Datei `start_project.bat`. Wählen Sie im Dropdown-Menü "Anzeige-Indikator" die Option "Einwohnerzahl". Die Karte rendert augenblicklich das Gitter samt automatischer Legende unten rechts.

Klicken Sie auf ein beliebiges Quadrat. Die Kachel wird mit einem fetten, permanenten anthrazitfarbenen Rahmen versehen. Das Dashboard links befüllt sich live mit den POI-Distanzen. Klicken Sie nun unten in der Sidebar auf die Buttons **MO**, **FR**, **SA**, **SO**. Die Taktlinie im Diagramm verschiebt sich interaktiv, während die Y-Achse starr blockiert bleibt, um die Angebotsschwankungen perfekt sichtbar zu machen.

8.2 Troubleshooting-Matrix

Symptom / Fehler	Ursache	Lösung
<code>CORS Missing Allow Origin (Status 500)</code>	Das Python-Backend ist im Hintergrund abgestürzt (z. B. durch eine veränderte DB-Tabellen-ID).	Schließen Sie das Uvicorn-Terminalfenster und starten Sie die <code>start_project.bat</code> komplett neu.
<code>ModuleNotFoundError: No module named 'gpd'</code>	Ein Import-Typo im Kopfbereich der Pipeline-Datei.	Öffnen Sie die <code>backend/pipeline.py</code> , löschen Sie die Zeile <code>import gpd as gpd</code> und speichern Sie ab.
Taktlinien verändern sich beim Button-Klick nicht.	Der Webbrowser führt noch eine alte Version der JavaScript-Datei aus seinem internen Cache aus.	Drücken Sie im Browser die Tastenkombination Strg + F5 (Hard Reload) oder nutzen Sie den Inkognito-Modus.